

DATA SCIENCE AND AI TOOLS FOR BEGINNERS

THE PROBLEM TO SOLVE

- You have a task that can be *automated with code*
- You want to know how an *AI assistant* can help

WHY AUTOMATION WITH CODE?

- **Repetitive tasks** can be automated to *save time and reduce errors*
- **Scalability:** Automation allows *scaling out in parallel* if needed
- **Scheduled tasks** can be automated to run at specific times *without human attention*
- **Judgment-based tasks** can be automated with *AI acting as your surrogate*
- **Complex tasks** can be automated by *building workflows incrementally*, with each step validated before moving on to the next
- **Reproducibility:** Automated workflows can be *reproduced exactly*, verified by others, and shared with others/the future self

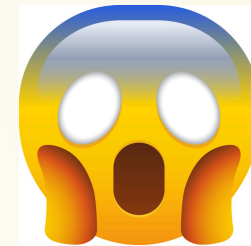
Code enables knowledge accumulation and transfer

AUTOMATION WITH CODE: EXAMPLES

- **Structured/Unstructured data collection**, such as database queries, web scraping
- **Summarization/Categorization** with large language models (LLMs) via API calls
- **Document/Image processing**, such as text extraction, color correction, watermarking

NO PROGRAMMING EXPERIENCE?

- “*I don’t know how to code!*”
 - **English** is the new programming language
- Your productivity is only limited by **how clear**
 - you *explain your problem* to AI
 - you *specify your requirements* for the solution
- A capable AI assistant will fill in the gaps in between
 - but you need to **guide** it with clear instructions and feedback



GITHUB ECOSYSTEM

GitHub Repository

Storage & Version Control



Your code lives here.

GitHub Codespaces

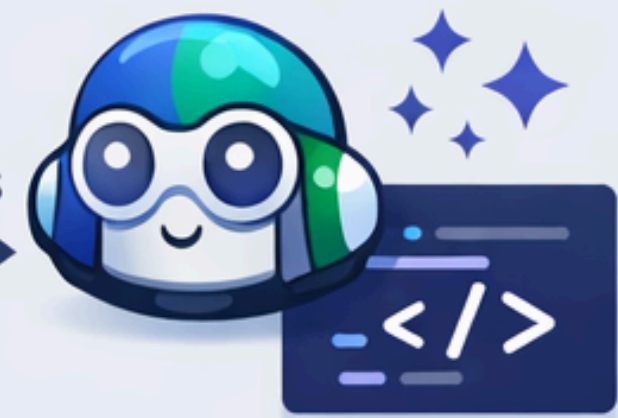
Cloud Coding Environment



Code from anywhere.

GitHub Copilot

AI Code Assistant



AI-powered suggestions.

opens

assists

WHY GITHUB, GITHUB CODESPACE, AND GITHUB COPILOT?

- **GitHub** provides spaces to *store, manage, and share* code with others
 - Code files of a project are organized in a *repository (repo)* with *version control* to track changes
- **GitHub Codespace** (built on **VS Code**) provides a *ready-to-use development environment* in the cloud, without requiring the setup of VS Code in a local machine
- **GitHub Copilot** is an *AI assistant* sitting inside Codespace/VS Code to
 - answer your questions (*Ask* mode)
 - formulate plans to implement your ideas (*Plan* mode)
 - act as your agent to execute the plans to generate code and verify the results (*Agent* mode)

BRANCHES OF A GITHUB REPO

- **Main branch:** The default branch of a repo, where the *stable version* of the code is stored
- **Feature/Development branch:** A branch created to develop a new feature or experiment with code changes, without affecting the main branch

IF AI CAN FORMULATE AND EXECUTE PLANS FOR ME, ...

- Why do I need to learn coding?



WHY STILL LEARN CODING?

- As of **today**, the AI models most of us can afford for heavy, daily use are ***still not smart enough*** to always deliver the required results without guidance
- The more you know, the better you can ***stay on top*** of everything
 - Otherwise, you won't notice any **wrong direction** AI might take you to — and won't be able to correct it in time

HOW USEFUL IS AI?

- Easy to use AI to deliver a *working solution*
 - Especially useful when
 - you don't know how to code, or
 - you don't have any precise requirements for the solution
- Not so easy to get AI to deliver a *solution exactly as you want*
 - Knowing how to give *directional guidance* while leveraging AI's capability to *fill in the gaps* is key to success

WHY VERSION CONTROL IS IMPORTANT

- Version control (by **Git**) tracks changes to your code over time
- To **commit** changes (i.e., save them for the long term), you must give a *commit message* to describe the changes made
- This sets a checkpoint that allows *reverting to a previous version* of the code,
 - protecting you from losing the original work after *failed experimentation* or accidental deletion by you or your **AI assistant**

MARIMO

- **marimo** is a *reactive* Python notebook
 - automatically *runs dependent cells* whenever a cell is run or a UI element is interacted with,
 - keeping code and outputs consistent.
- Especially *friendly to beginners* due to:
 - using **uv** to handle packages installation and environment setup *under the hood*

PAIN POINTS OF USING PYTHON

- The power of Python comes from its **ecosystem of libraries** (aka. packages) that provide pre-built functions for various tasks
 - Packages are often developed by different authors, and may have **different versions** with **incompatible dependencies**, let alone the **Python version** itself
- Traditional installation methods use `pip` or `conda` to install packages, which can lead to **dependency conflicts** and **version mismatches**, especially when multiple projects require different versions of the same package
 - The solution is to use **virtual environments** to isolate dependencies for each project, but this can be cumbersome and error-prone for beginners
 - The maintenance of virtual environments results in duplicated packages and **wasted disk space**, especially when multiple projects share similar dependencies

uv AS THE RESCUE

- ensure the correct versions of packages are installed and used
- avoid duplicated installations and wasted disk space by
 - using **symlinks** to point to the shared packages instead of copying them

WHY MARIMO?

- Other often praised features include:
 - Built-in interactive elements
 - Directed Acyclic Graph (DAG)-based
 - handles **code execution order** automatically, so you don't have to worry about the order of code cells
 - Dual-mode execution
 - can be run in an **interactive** interface and in a **shell**, bridging the gap between exploratory **notebook-based** and production-ready **script-based** workflows
 - Useful details provided in the **notebook interface** than Jupyter notebook when exploring data with pandas

MARIMO DON'TS

- Do NOT re-define an object in another cell
 - Variable names and function names must not be re-used
 - Or, will cause an error like this:

This cell wasn't run because it has errors

This cell redefines variables from other cells.

SP500_data was also defined by:

cell-75

Why can't I redefine variables? marimo requires that each variable is defined in just one cell. This constraint **enables** reactive and reproducible execution, **arbitrary cell reordering**, seamless UI elements, execution as a script, and more.

Try merging this cell with the mentioned cells or wrapping it in a function. Alternatively, *rename variables to make them private to this cell by **prefixing them with an underscore** (e.g. `_SP500_data`).*

Learn more at our docs ↗: <https://docs.marimo.io/guides/reactivity/#multiple-definition-error>

`.devcontainer` IN CODESPACE

- For marimo to be beginner-friendly and *work out of the box*,
 - marimo and its VS Code extension must be preinstalled in a Codespace
- The magic comes from configuration files in the `.devcontainer` folder,
 - which specify the packages and extensions to install and other settings for the Codespace environment

CODE EXAMPLES (IN MARIMO)

- Unstructured data collection & Document processing

UNSTRUCTURED VS STRUCTURED DATA

- **Structured data**

- follows a consistent pattern and thus can be organized in a fixed format,
 - referred to as a **schema** that defines each field's data type (e.g., integer, text string, double[-precision]/floating-point, date, etc)

- Example:

ID	Name	Weight (kg)	Email	Join_Date
1324	John D	76.1	john@email.com	2024-01-01
2846	Mary S	65.5	mary@email.com	2024-01-15

- **Unstructured data**

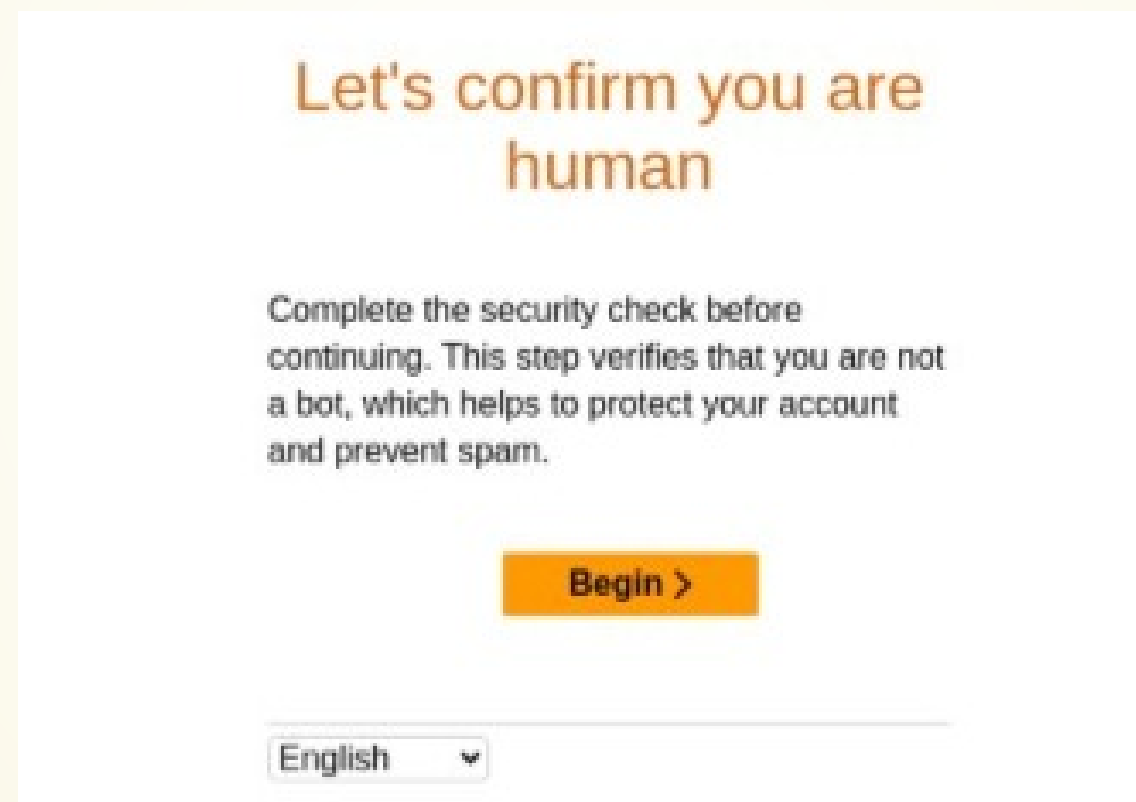
- is inconvenient to organize into a schema, e.g.,
 - Instagram posts - can **vary in the number** of images
 - company reports - can **differ substantially in length**

APPLICATION

- **Collecting sustainability reports from company websites**
 - **What are the challenges (and solutions)?**

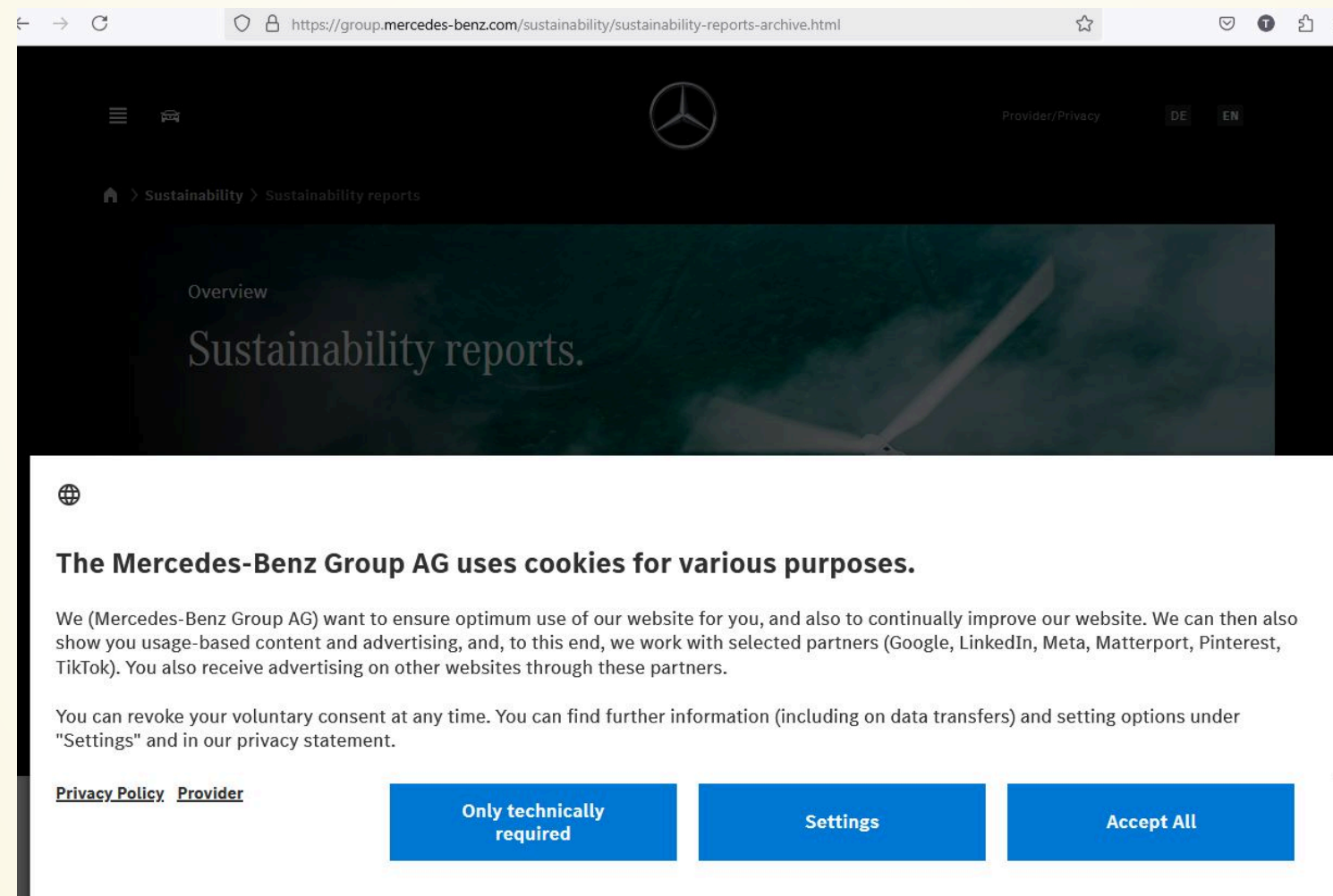
HUMAN VERIFICATION TRIGGERED BY BOT DETECTION

- blocks access to a website completely



COOKIES POPUP

- blocks access to the information on a webpage



OTHER CHALLENGES

- Location of target data
 - often *unknown in advance*
- Downloading multiple data files
 - a tedious task that may lead to *omissions* if done manually
- Extracting texts from pdf files for analysis
 - might need to use an **optical character recognition (OCR)** approach if a file is *not text-searchable*

LET'S DIVE IN

- marimo notebooks in **Workshop repo**
 - `1acceptNstoreCookies.py`
 - `2collect_urls.py`
 - `3DLnExtract_OCR.py`



OVERVIEW OF THE MARIMO NOTEBOOKS

- `1acceptNstoreCookies.py`
 - bypasses **bot detection** by a target website
 - handles any **cookie consent banners** that appear (the pop-ups asking you to accept or reject cookies)
 - saves the resulting **cookies** and **local storage** so that later scripts can reuse them
 - collects the **list of URLs** found on the website's landing page
- May be executed with the **terminal command**:
 - `python3 1acceptNstoreCookies.py`

1a. AVOID BOT DETECTION

- Define settings
 - website to visit, topics to filter URLs by
- Launch a browser (**Playwright** / Chromium)
 - CONTEXT 1 — no evasion:
 - Open a page with no user agent → gets blocked by the website
 - Take screenshot → [screenshot_BotDetected.png](#)
(confirms the block is real)
 - CONTEXT 2 — evasion:
 - Set a realistic user agent string and use tricks to hide automation flags **as though** the browser was used by a **human**
 - Open a fresh page → website now loads correctly

1b. ACCEPT & STORE COOKIES

- ■ CONTEXT 2 — evasion (cont'd):
 - **cookie banner visible** → `screenshot_CookiesPopup.png`
 - Click the “Accept All” button on the cookie banner
 - **banner gone** → `screenshot_CookiesAccepted.png`
 - Extract all URLs from the **landing page** → `urls_raw.csv`
 - **Filter URLs by topic keywords** → `urls_filtered.csv`
 - Save browser cookies & local storage
→ `cookies.json, localStorage.json`

OVERVIEW OF THE MARIMO NOTEBOOKS (CONT'D)

- `2collect_urls.py`
 - crawls a website starting from a set of **seed URLs**
 - follows links up to a configurable **depth**
 - collects all URLs matching specified **keywords** (e.g., sustainability, ESG)
- May be executed with the **terminal command**:
 - `python3 2collect_urls.py`

2a. WEBCRAWLING TO COLLECT URLS

- Define settings
 - website to crawl
 - only URLs sharing the **highest two-level domain** will be visited (e.g., ['siemens', 'com'])
 - keywords to look for, **how deep** to follow links
 - **maximum run time** to avoid webcrawling indefinitely
- Read candidate URLs from **urls_filtered.csv** (previously saved)
 - Remove URLs with **redundant sub-paths** → **seedURLs.csv**
 - e.g., **/sustainability/reports/2023** is nested inside **/sustainability/**
 - Load saved browser **cookies** and **local storage** (so the crawler looks like a real user)

2b. WEBCRAWLING TO COLLECT URLS

- Launch a browser (**Playwright** / Chromium)
 - For each **seed URL**:
 - Visit the page
 - **Collect all links** on that page
 - For each link that matches a keyword and is **valid for visit** (e.g., not a [mailto:](#) or [pdf](#))
 - Visit that page and repeat **recursively** up to the **allowed depth**
- Save all URLs discovered → [allURLs.csv](#)
(and [allURLs_df.csv](#) with Filename and Extension columns)
 - Save PDF links only → [pdfURLs.csv](#)
 - Save PDFs matching topics → [pdfscreenedURLs.csv](#)
 - Save pages actually visited → [visitedURLs.csv](#)
 - Will not be visited again in future runs of the crawler

OVERVIEW OF THE MARIMO NOTEBOOKS (CONT'D)

- `3DLnExtract_OCR.py` automates a PDF data collection pipeline:
 1. **Download** — fetch PDF sustainability reports from a list of screened URLs
 2. **Analyse** — count occurrences of user-defined keywords on each page of the PDFs
 3. **Extract** — save pages with the keywords as individual single-page PDFs

3a. DOWNLOAD AND EXTRACT RELEVANT PAGES

- Define settings
 - **year** to target, **keywords** to search for in PDFs
 - fraction of reports to download (for demo purpose)
- Create output folders
 - `docArchive/DLdocs/` (for downloaded PDFs)
 - `docArchive/pagesExtracted/` (for extracted pages)
- Read the screened PDF URL list from `pdfscreenedURLs.csv` (previously saved)
 - Keep only the URLs containing the target year (e.g. `2019`)
 - Load saved browser **cookies** (so the downloader looks like a real user)

3b. DOWNLOAD AND EXTRACT RELEVANT PAGES

- For each URL in the filtered list (up to the chosen fraction):
 - Check the **download ledger** `df_DL.csv` → Skip if already downloaded
 - Otherwise, download and save the PDF to `DLdocs/`
 - **Update** the download ledger `df_DL.csv` accordingly
 - Check internal **page numbering** of the downloaded PDF
 - if broken → fix it after backing up the original to `bad/` subfolder
 - Extract text from the PDF based on PDF type:
 - if **text-searchable** → use `PyMuPDF`
 - if scanned → use **OCR** tools to recognize text
 - **Count occurrences** of the keywords page-by-page
 - Record and save `[page number, keyword count]` pairs in `df_DL.csv`
- For each page on the list of `[page number, keyword count]` pairs:
 - Extract and save the page to `pagesExtracted/`

GETTING **AI** ASSISTANCE

INLINE AUTOCOMPLETION

- Inline suggestions for **code completion**, documentation, and code snippets
 - Also, useful for *developing slides* and *writing documents* (e.g., in Quarto/Markdown) in editors
- To *reject* an inline suggestion,
 - pressing **Esc** to express “*Leave me alone for now*”

ASK VS. PLAN MODE

- **Ask mode:** Q&A conversation in a chatbox — like most people's first experience with chatGPT
- **Plan mode:** Describe a problem for AI to formulate a solution plan,
 - which you can *review and agree on before implementation*
- Downsides of *using Plan mode for an Ask-mode question*
 - **Wrong output shape** — producing a plan when you just want a direct explanation
 - **More overhead** (thorough upfront investigation of the codebase)
 - wasted effort when you just want an answer
 - **Nudging toward Start Implementation**
 - even when you don't intend to make code changes

PLAN VS. AGENT MODE

- Under **Plan mode**, the AI assistant is *not allowed to change* any code
- **Agent mode**: Can *autonomously generate/alter/delete* code as needed to achieve the goal specified by you (or in the agreed plan)
 - Depending on the configuration, may still need your permissions on actions with **significant consequences**,
 - e.g., using certain tools such as **bash** commands, accessing the internet, or sending emails

IMPORTANCE OF CONTEXT WINDOW SIZE

- The **context window** size determines the amount of information that can be considered at once by an AI model
- The invisible **system prompt** for guiding the model's behavior and responses can take up a significant portion of the context window,
 - leaving less space for user input (**prompts**) and model output (**responses**)
- A larger context window allows the model to understand and generate more ***coherent and contextually relevant responses***, especially for complex tasks involving long documents or conversations.
- Models with smaller context windows handle context exhaustion through **compression/compaction** (i.e., truncating or summarizing the input)
 - May lead to loss of important input information and less accurate output

RULES OF THUMB

- Avoid long conversations that exceed a model's context window
- At least **1M tokens** context window is necessary for a series of related complex tasks emerging from a long conversation
- **At least 200K tokens** context window is needed for serious work other than simple tasks
- If **two rounds** of prompt-and-response aren't getting you closer to your goal (e.g., fixing a bug),
 - consider switching to a more capable (likely more expensive) model but **more expensive models are not necessarily better**
- Experiment with different models to find out which ones work well for particular tasks and requirements

OPENCODE / GITHUB COPILOT / CODEX / CLAUDE CODE

- Why **OpenCode**?
 - **Open-source** project avoiding vendor lock-in
 - unlike **Claude Code** and **GitHub Copilot**, which are closed-source
 - Similarly **wide support for LLM providers** as GitHub Copilot
 - unlike **Codex**, which sends some tool calls in OpenAI-specific formats — non-OpenAI LLMs don't always work reliably even with translation
 - **Comparable harness performance** to other agents for third-party LLMs
- **OpenCode Zen** has a few free models with generous **rate limits** and context window sizes for users to get started without paying anything
 - e.g., *MiMo V2.5 Free* with 200K tokens context window
- **OpenCode Go** offers a low-cost monthly subscription plan with a range of serious models suitable for more complex tasks
 - e.g., *DeepSeek V4.1 Flash* with 1M tokens context window

LLM ACCESS VIA API

- Minimal example: `4MinExample_LLMapi.py`
 - input data: `4MinExample_Notes.parquet`
 - output results: `4MinExample_Coded.parquet`

POWER OF GPU ACCELERATION

PLATFORMS WITH GPU SUPPORT

- Google **Colab** has Jupyter Notebook as its GUI with Gemini as its AI assistant
 - supports Python/R/Julia runtime
 - free-tier account:
 - can run notebooks for at most 12 hours
 - has very limited access to GPU with other resource limits
- **Lightning.ai** has VS Code and JupyterLab as its GUIs (VS Code interface has GitHub Copilot)
 - free-tier account offers:
 - 1 free active Studio (for every 4 hours)
 - 80 hours of free-trial GPU access

LIGHTNING.AI

Free

\$0

Students, researchers, hobbyists

Choose Free

💰 up to 30 credits

Key features

- ✓ 1 free active Studio (4 hour restarts)
- ✓ 32 core CPU Studios
- ✓ Single GPUs (L40S, A100, H100, H200...)
- ✓ 80% off on interruptible (spot)
- ✓ Up to 2 concurrent GPUs
- ✓ Persistent storage (50 GB limit)

- ✓ 15 req/min to our model APIs
- ✓ 120,000 tok/min to our model APIs
- ✓ Connect any local IDE or SSH
- ✓ Unlimited background execution
- ✓ Multiplayer live collaboration
- ✓ Use private and public models
- ✓ Access optimized Studios
- ✓ Automate with our SDK
- ✓ Pay as you go

- How much can a GPU accelerate?
 - Let's look at an application example with **natural language processing (NLP)**

BIGRAM CLOUD

- *Word cloud*
 - visual representation of text data with
 - each word's **size** indicating its **frequency/importance**
 - *Bigram cloud* extends the concept to pairs of words (*bigrams*) instead of single words
- Application example:
 - What **risk factors** are mentioned in *Item 1A* of *Magnificent 7*'s 10-K filings (2015 vs. 2025)?
 - AAPLE, MSFT, GOOGL, AMZN, NVDA, META, and TSLA

IDENTIFYING BIGRAMS IN TEXT

- **Stop words** (e.g., “of”, “the”, “is”, “and”)
 - removed to focus on more meaningful content
- Older approaches focused on **stemmed words**
 - reduced to their *root form* (e.g., “vis” for vision, visions, visionary, and visibility)
- Newer approaches use **lemmatized words**
 - reduced to their *base form* (e.g., “vision” for vision and visions; “visionary” for visionary)
- Lemmatization is computationally intensive, which benefits greatly from **GPU acceleration**

STUDIO ON LIGHTNING.AI

- **Studio** in Organization on *Lightning.ai*
 - is like **Codespace** on GitHub
- See the shared Studio template: **BigramCloud_GPUorCPU**

STRENGTHS OF R ECOSYSTEM

R SYNTAX

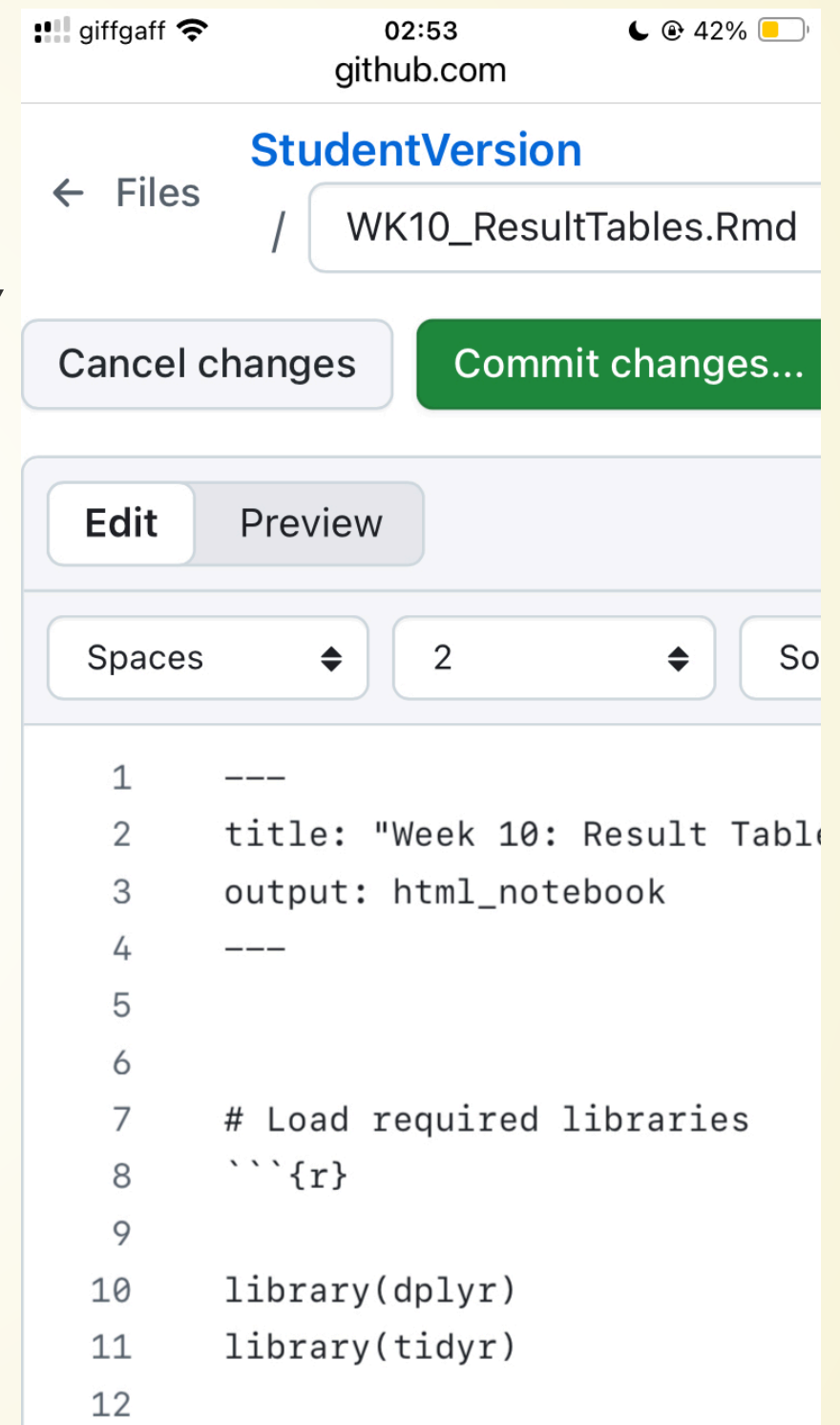
- Often more *concise* and *expressive* for data manipulation and analysis tasks

```
WK10_ResultTables.Rmd
28
29 # Merge datasets
30 ```{r}
31 You, yesterday • add more Week 10 files
32 ESG_data_new <- ESG_data |>
33   mutate(date = as.character(date)) |>
34   left_join(ESGscore_TobinsQ, by = c("instrument" = "tic", "date" = "datadate")) |>
35   mutate(
36     at_pos = ifelse(at > 0, at, NA), # Replace non-positive 'at' values with NA
37     ceq_pos = ifelse(ceq > 0, ceq, NA), # Replace non-positive 'ceq' values with NA
38     q = (prcc_f * csho + dlitt + dlc) / at_pos, # Tobin's Q calculation
39     ESGscore = esg_metric,
40     Firm_Size = log(at),
41     Leverage = (dlitt + dlc) / ceq_pos,
42     Year = fyear,
43     Industry = sich,
44   ) |>
45   select(source, q, ESGscore, Firm_Size, Leverage, Year, Industry) |>
46   drop_na() # Drop any remaining rows with NA values in the selected columns
47
48 ESG_data_B <- ESG_data_new |>
49   filter(source == "Provider_B") |>
50   select(-source) # Remove 'source' column as it's no longer needed
51
52 ESG_data_A <- ESG_data_new |>
53   filter(source == "Provider_A") |>
54   select(-source)
55
```

```
WK10_ResultTables.py > Merge and engineer features > df = (
Generate + Code + Markdown | Run All Clear All Outputs | Run stale ... Select Kernel
1 # Merge and engineer features
Generate + Code + Markdown
Stale markdown
1
2 df = (
3   esg_data
4   .assign(date=lambda x: x["date"].astype(str))
5   .merge(esg_tq, left_on=["instrument", "date"], right_on=["tic", "datadate"],
6         how="left")
7   .assign(
8     at_pos = lambda x: x["at"].where(x["at"] > 0),
9     ceq_pos = lambda x: x["ceq"].where(x["ceq"] > 0),
10    q = lambda x: (x["prcc_f"] * x["csho"] + x["dlitt"] + x["dlc"]) / x
11    ["at_pos"],
12    ESGscore = lambda x: x["esg_metric"],
13    Firm_Size = lambda x: np.log(x["at"]),
14    Leverage = lambda x: (x["dlitt"] + x["dlc"]) / x["ceq_pos"],
15    Year = lambda x: x["fyear"],
16    Industry = lambda x: x["sich"],
17  )
18  [
19    ["source", "q", "ESGscore", "Firm_Size", "Leverage", "Year", "Industry"]
20  ].dropna()
21
22 df_B = df[df["source"] == "Provider_B"].drop(columns="source").reset_index(drop=True)
23 df_A = df[df["source"] == "Provider_A"].drop(columns="source").reset_index(drop=True)
24
25 # Summary statistics
26 df_B.describe()
Stale Python (marimo)
```

RMARDOWN NOTEBOOKS

- **Plain-text files**
 - easy to version-control
 - readable and editable even *on the go with only a phone*
- Structured in **JSON** format, *Jupyter* notebooks are
 - difficult to version-control effectively
 - **not** as *human-readable* or *edit-friendly* outside of the notebook interface
 - Similarly for *marimo* notebooks, despite being Python scripts (in plain text)
 - Next slide shows the **source code structures** of Jupyter and marimo notebooks



The screenshot shows a mobile application interface for editing a markdown file. At the top, the status bar shows 'giffgaff' as the carrier, signal strength, Wi-Fi, time '02:53', and battery level '42%'. Below the status bar, the app header shows 'github.com' and a back arrow. The main title is 'StudentVersion' in blue. Below the title, there's a breadcrumb 'Files / WK10_ResultTables.Rmd'. Two buttons are visible: 'Cancel changes' and 'Commit changes...'. Below these, there are tabs for 'Edit' and 'Preview'. Under the tabs, there's a dropdown menu for 'Spaces' set to '2'. The main content area shows a code editor with line numbers 1 through 12. The code is a markdown snippet with a title and some Python code for loading libraries.

```
1 ---
2 title: "Week 10: Result Table
3 output: html_notebook
4 ---
5
6
7 # Load required libraries
8 ```{r}
9
10 library(dplyr)
11 library(tidyr)
12
```

UNLIKE JUPYTER/MARIMO NOTEBOOKS

```
WK10_ResultTables.ipynb > ...
2 "cells": [
56 {
57   "cell_type": "code",
58   "execution_count": 8,
59   "metadata": {},
60   "outputs": [],
61   "source": [
62     "# Merge and engineer features\n",
63     "df = (\n",
64     "    esg_data\n",
65     "    .assign(date=lambda x: x[\"date\"].astype(str))\n",
66     "    .merge(esg_tq, left_on=[\"instrument\", \"date\"], right_on=[\"tic\", \"datadate\"],\n",
67     "    how=\"left\")\n",
68     "    .assign(\n",
69     "        at_pos = lambda x: x[\"at\"].where(x[\"at\"] > 0),\n",
70     "        ceq_pos = lambda x: x[\"ceq\"].where(x[\"ceq\"] > 0),\n",
71     "        q = lambda x: (x[\"prcc_f\"] * x[\"csho\"] + x[\"dltt\"] + x[\"dlc\"] / x\n",
72     "        [\"at_pos\"],\n",
73     "        ESGscore = lambda x: x[\"esg_metric\"],\n",
74     "        Firm_Size = lambda x: np.log(x[\"at\"]),\n",
75     "        Leverage = lambda x: (x[\"dltt\"] + x[\"dlc\"] / x[\"ceq_pos\"],\n",
76     "        Year = lambda x: x[\"fyear\"],\n",
77     "        Industry = lambda x: x[\"sich\"],\n",
78     "    )\n",
79     "    [[\"source\", \"q\", \"ESGscore\", \"Firm_Size\", \"Leverage\", \"Year\", \"Industry\"]]\n",
80     "    .dropna()\n",
81     "    )\n",
82     "df_B = df[df[\"source\"] == \"Provider_B\"].drop(columns=\"source\").reset_index(drop=True)\n",
83     "df_A = df[df[\"source\"] == \"Provider_A\"].drop(columns=\"source\").reset_index(drop=True)\n",
84   ],
85   "cell_type": "code",
86   "execution_count": 9,
```

Jupyter

```
WK10_ResultTables.py > ...
38
39
40 @app.cell
41 def _(esg_data, esg_tq, np):
42
43     df = (
44         esg_data
45         .assign(date=lambda x: x["date"].astype(str))
46         .merge(esg_tq, left_on=["instrument", "date"], right_on=["tic", "datadate"], how="left")
47         .assign(
48             at_pos = lambda x: x["at"].where(x["at"] > 0),
49             ceq_pos = lambda x: x["ceq"].where(x["ceq"] > 0),
50             q = lambda x: (x["prcc_f"] * x["csho"] + x["dltt"] + x["dlc"]) / x["at_pos"],
51             ESGscore = lambda x: x["esg_metric"],
52             Firm_Size = lambda x: np.log(x["at"]),
53             Leverage = lambda x: (x["dltt"] + x["dlc"]) / x["ceq_pos"],
54             Year = lambda x: x["fyear"],
55             Industry = lambda x: x["sich"],
56         )
57         [["source", "q", "ESGscore", "Firm_Size", "Leverage", "Year", "Industry"]]
58         .dropna()
59     )
60     df_B = df[df["source"] == "Provider_B"].drop(columns="source").reset_index(drop=True)
61     df_A = df[df["source"] == "Provider_A"].drop(columns="source").reset_index(drop=True)
62     return df_A, df_B
63
64
65 @app.cell
66 def _(df_B):
67     # Summary statistics
68     df_B.describe()
69     return
70
71
```

marimo

GLOBAL LINE NUMBER REFERENCE

- Jupyter notebooks (and marimo in the notebook interface) **cannot show globally unique line numbers** for the entire notebook,
 - hard to reference specific code lines for **debugging** and documentation
- Not a problem for Rmarkdown notebooks as plain-text files

STATISTICAL/ECONOMETRIC ANALYSIS

- Packages *specialized* for statistical analysis and econometrics are more likely to be available in R than Python
- For example,
 - *difference-in-differences* (DiD) analysis is an important methodology for measuring **causal effects**
 - This **DiD** website maintains a comprehensive list of packages for DiD analysis:
 - **16 packages in R vs. 11 in Python**

EXPANDING YOUR DS TOOLKIT

- No single tool solves every problem
 - Understanding the “right tool for the job” is a key skill
- The landscape of Data Science and AI **changes rapidly**
 - Knowing where to find resources is essential for continuous **self-learning and exploration**
- For example,
 - see the newsletter [The Batch](#) and courses on [DeepLearning.AI](#)
 - also, [AlphaSignal](#)’s daily newsletter on AI, machine learning and cutting-edge language models

OPEN-SOURCE PACKAGES ON GITHUB

- Powerful packages are freely available on GitHub
 - The challenge: Buried among numerous **poorly maintained/poorly documented** packages on GitHub



ADOPTING OPEN-SOURCE RESOURCES

- **Availability** of open-source packages / data?
 - free vs. freemium vs. paid
- **Reliance** considerations:
 - Abandoned projects?
 - **Organizational backing** (e.g., **polars**)
vs. Individual developer
 - **Active development**:
 - Last commit
 - Watch | Custom | Releases, Issues, Security alerts

ADOPTING OPEN-SOURCE RESOURCES

- Reliance considerations (cont'd):
 - **Maintenance** commitment:
 - Issues tab: Open vs. Closed
 - **Usefulness**
 - Stars, Used by, Contributors
 - **Star History** graph, **GitHub Trending** ranking
 - **Documentation** quality
 - README, docs/doc, Vignettes, Wiki, Website

POLARS VS. PANDAS

- Pandas requires RAM to be **at least 2 times** of the dataset size to work
- Polars is
 - **memory-efficient** due to various approaches, including
 - **Streaming**: natively process data in optimized batches or chunks for **big data** (i.e., too big to fit into RAM)
 - built on **Rust** — known for its speed and safety
 - **cross-platform**: including R and various other environments
- Polars achieves high performance through **parallel execution** and
 - **lazy evaluation**
 - e.g., if you load a file with 100 columns but only use 3, Polars reads only those 3 columns into memory
(by first analyzing the code to determine which are actually needed)

THE MONTH AFTER THIS WORKSHOP

- Recommendation:
 - ***Do not wait*** until you have a serious task/project to start using AI
 - Simply keep on integrating AI into your workflow on a ***daily basis***
 - e.g., start a low-stake toy project that would not matter even if it fails
- Up-to-date information you learned today can ***become obsolete in a few months***
- Only when AI has become an ***inextricable*** part of your workflow will you be
 - motivated to keep up with the latest developments
 - able to accumulate experience and leverage it successfully to solve serious problems

MY TOY PROJECT



GitHub Repo Activity — last 30 days (Forums: 116/384; Focus: 35/373; Others: 158/1646)

1 message

Tue, Sep 22, 2026 at 3:08 AM

GitHub Repo Activity

last 30 days · Forums + Focus + Others

Forums (Inbox)

Last 30 days · 116 repos · 384 notifications

#	REPO	CREATOR	FREQ	LATEST	DATE
1	agentfield	Agent-Field	67	Release v0.1.140 - v0.1.140	2026-09-21
2	QwenPaw	agentscope-ai	15	Pre-release v2.2.2-beta.3 - v2.2.2-beta.3	2026-09-20
3	marimohub	marimo-team	14	Release v0.4.8 - v0.4.8	2026-09-21
4	deepseek-harness	deepseek-ai	14	Pre-release dsh-v0.1.6-alpha.2 - v0.1.6-alpha.2	2026-09-17
5	coolify	coollabsio	13	Release v4.3.23 - v4.3.23	2026-09-18
6	oh-my-opencode-slim	alvinunreal	11	Release v2.2.22 - v2.2.22	2026-09-19
7	vscode-R	REditorSupport	10	Pre-release latest - Development Build	2026-09-21
8	unsloth	unslothai	10	Pre-release v0.1.811-beta - Docker + Multi User +...	2026-09-18
9	LangAlpha	ginlix-ai	8	Release v2026.09.07 - v2026.09.07	2026-09-07
10	pdf-inspector	firecrawl	7	Release v1.23.0 - 1.23.0 — Text-layer repairs	2026-09-22
11	pi	earendil-works	7	Release v0.87.0 - v0.87.0	2026-09-21
12	changedetection.io	dgtlmoon	7	Release 0.60.7 - 0.60.7	2026-09-17
13	VoiceStudio	debpalash	6	Release v0.5.5 - v0.5.5 — Cross-language cloning,...	2026-09-22

THANK YOU AND BEST WISHES!